

Assignment No. 1 Program

Implementation of a robust Java calculator program using OOP concepts, conditional logic, looping constructs, and exception handling.

```
import java.util.InputMismatchException;
import java.util.Scanner; public class
Calculator {
    static double addition(double a, double b) {
return a + b;
    }
    static double subtraction(double a, double b) {
return a - b;
    }
    static double multiplication(double a, double b) {
return a * b;
    }
    static double division(double a, double b) {
if (b == 0) {
        throw new ArithmeticException("Division by zero is not allowed.");
    }
return a / b;
    }
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);          int
choice;
        boolean continueCalc = true;
do {
    try {
        System.out.println("\n--- Calculator Menu ---");
        System.out.println("1. Addition");
        System.out.println("2. Subtraction");
        System.out.println("3. Multiplication");
        System.out.println("4. Division");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
choice = sc.nextInt();          if (choice == 5) {
```

```

        System.out.println("Exiting Calculator. Thank you!");
break;

    }
    System.out.print("Enter first number: ");
double num1 = sc.nextDouble();
    System.out.print("Enter second number: ");
double num2 = sc.nextDouble();
double
result;
    switch (choice) {
case 1:

        result = addition(num1, num2);
        System.out.println("Result = " + result);
break;
        case 2:
            result = subtraction(num1, num2);
System.out.println("Result = " + result);
break;
        case 3:
            result = multiplication(num1, num2);
System.out.println("Result = " + result);
break;
        case 4:
            result = division(num1, num2);
            System.out.println("Result = " + result);
break;
        default:
            System.out.println("Invalid choice! Please select 1 to 5.");
    }
} catch (InputMismatchException e) {
    System.out.println("Invalid input! Please enter numeric values
only.");
    sc.nextLine(); // clear buffer
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
}
    } while (continueCalc);
sc.close();
}
}

```

Assignment No. 2 Program

Development of a Java program for an E-commerce order processing system using

constructors, dynamic input, and conditional logic.

```
import java.util.Scanner;

class Product {
    String name;
    double price;
    int quantity;

    // Constructor 1: Default Product
    Product() {
        name = "Unknown";
        price = 0.0;
        quantity = 0;
    }

    // Constructor 2: Initialize with name and price
    Product(String name, double price) {
        this.name = name;
        this.price = price;
        quantity = 1; // default quantity
    }

    // Constructor 3: Initialize with full details
    Product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    double getTotalCost() {
        return price * quantity;
    }

    // Static method to apply discount
    static double applyDiscount(double total) {
        if (total > 15000)
            return total-total * 0.25;
        else if (total > 10000)
            return total-total * 0.20;
        else if (total > 5000)
            return total-total * 0.15;
        else
            return total;
    }
}
```

```

        // Method to display product details
void displayProduct() {
    System.out.println( name + " | Price: " + price + " | Quantity: " + quantity
+ " | Total: " + price * quantity );
    } }
public class Ecommerce {
    public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
        Product p1 = new Product("Powerbank ", 800);
        Product p2 = new Product("Headphones", 150, 2);
        // User input
        System.out.print("Enter product name: ");
        String name = sc.nextLine();
        System.out.print("Enter product price: ");
double price = sc.nextDouble();
        System.out.print("Enter quantity: ");          int
quantity = sc.nextInt();
        Product userProduct = new Product(name, price, quantity);
        // Display order summary
        System.out.println("\nOrder Summary:");
p1.displayProduct();
p2.displayProduct();
userProduct.displayProduct();          //
Calculate total and discount          double
totalCost = p1.getTotalCost()
+ p2.getTotalCost()
                + userProduct.getTotalCost();
double finalAmount = Product.applyDiscount(totalCost);
        System.out.println("\nTotal Cost Before Discount: " + totalCost);
        System.out.println("Final Price After Discount: " + finalAmount);
        System.out.println("Thank you for shopping!");          sc.close();
    }
}

```

Assignment No. 3 Program

Implementation of a Java program to demonstrate method overloading.

```
import java.lang.Math;

public class OverloadingDemo {

    // Overloaded method to compute power
    int power(int base, int exp) {          int
result = 1;
        for (int i = 1; i <= exp; i++) {
result *= base;
        }
    return result;
    }

    double power(double base, int exp) {
double result = 1;          for (int i =
1; i <= exp; i++) {          result *=
base;
        }
    return result;
    }

    // Overloaded method to compute absolute value
    int absolute(int num) {
        return Math.abs(num); // static Math method
    }

    double absolute(double num) {
return Math.abs(num);
    } }

class TestOverloadingDemo {
    public static void main(String[] args) {
        OverloadingDemo c = new OverloadingDemo();
        System.out.println("Power (int): " + c.power(2, 3));
        System.out.println("Power (double): " + c.power(2.5, 3));
        System.out.println("Absolute (int): " + c.absolute(-10));
        System.out.println("Absolute (double): " + c.absolute(-15.75));
    }
}
```

Assignment No. 4 Program

Implementation of a Java program for a Library Management System using static fields

and methods to manage book records.

```
import java.util.Scanner;

class Book {
    String title;
    boolean isIssued;

    static int totalBooks = 0; // Static field to track total books
    // Constructor to initialize book
    public Book(String title) {
        this.title = title;          this.isIssued
        = false;
        totalBooks++; // Increment total book count
    }

    // Method to issue a book
    public void issueBook() {
        if (!isIssued) {
            isIssued = true;
            System.out.println("Book '" + title + "' has been issued.");
        } else {
            System.out.println("Book '" + title + "' is already issued.");
        }
    }

    // Method to return a book
    public void returnBook() {
        if (isIssued) {
            isIssued = false;
            System.out.println("Book '" + title + "' has been returned.");
        } else {
            System.out.println("Book '" + title + "' was not issued.");
        }
    }

    // Method to display book details
    public void displayBook() {
```

```

        System.out.println(title + " | " + (isIssued ? "Issued" : "Available"));
    }

    // Static method to display total books
    public static void displayTotalBooks() {
        System.out.println("Total Books in Library: " +
            totalBooks);
    }
}

// Main class
public class LibrarySystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Book[] library = new Book[5]; // Array to store books
        int count = 0; // Number of books added        while (true) {
            System.out.println("\nLibrary Management System");
            System.out.println("1. Add Book");
            System.out.println("2. Issue Book");
            System.out.println("3. Return Book");
            System.out.println("4. View Books");
            System.out.println("5. Check Total Books");
            System.out.println("6.Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline switch
            (choice) {
                case 1: // Add Book
                    System.out.print("Enter Book Title: ");

                    String title = scanner.nextLine();
                    library[count] = new Book(title);
                    count++;
                    System.out.println(" Book added successfully.");
                    break;
                case 2: // Issue Book
                    System.out.print("Enter Book Number to Issue (1-"
                        + count + "): ");
                    int issueNum = scanner.nextInt();
                    library[issueNum - 1].issueBook();
                    break;
                case 3: // Return Book
                    System.out.print("Enter Book Number to Return (1-" + count + "):
");

```

```
        int returnNum = scanner.nextInt();
library[returnNum - 1].returnBook();
break; case 4: // View Books
        System.out.println("\nLibrary Books:");
for (int i = 0; i < count; i++) {
System.out.print((i + 1) + ". ");
library[i].displayBook();
        }
break; case 5: // Check Total Books
Book.displayTotalBooks();
break;
case 6: // Exit
        System.out.println("Exiting the system. Thank you!");
scanner.close();                return;
default:  System.out.println("Invalid choice. Try again.");
        }
    }
}
```

Assignment No. 5 Program

Development of a Java program to perform various operations on array, including displaying elements, finding the maximum and minimum elements, calculating the sum and average of elements, and searching for a specific element within the array.

```
import java.util.Scanner;

class ArrayMenu {
    int arr[];
    int n;

    ArrayMenu (int size) {
n = size;
        arr = new int[n];
    }

    void inputArray() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }

    void display() {
        System.out.println("Array Elements:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }

    void findMax() {
        int max = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        System.out.println("Maximum Element: " + max);
    }

    void findMin() {
        int min = arr[0];
        for (int i = 1; i < n;
```

```

i++) {          if (arr[i] < min) {
min = arr[i];
        }
    }
    System.out.println("Minimum Element: " + min);
}    void findSum() {          int
sum = 0;          for (int i = 0; i <
n; i++) {          sum += arr[i];
    }
    System.out.println("Sum of Elements: " + sum);
}    void
findAverage() {
int sum = 0;

    for (int i = 0; i < n; i++) {
sum += arr[i];
    }
    double avg = (double) sum / n;
    System.out.println("Average of Elements: " + avg);
}
void searchElement() {
    Scanner sc = new Scanner(System.in);
System.out.print("Enter element to search: ");          int
key = sc.nextInt();

}
}

```

```

public class ArrayOperations {    public
static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter array size: ");
int size = sc.nextInt();
    ArrayMenu obj = new ArrayMenu (size);
obj.inputArray();

```

```

int choice;
do {
    System.out.println("\n--- Array Operations Menu ---");
    System.out.println("1. Display Array");
    System.out.println("2. Find Maximum");
    System.out.println("3. Find Minimum");
    System.out.println("4. Find Sum");
    System.out.println("5. Find Average");
    System.out.println("6. Search Element");
    System.out.println("7. Exit");
    System.out.print("Enter your choice: ");
    choice = sc.nextInt();
    switch (choice) {
        case 1:
            obj.display();
            break;
        case 2:
            obj.findMax();
            break;
        case 3:
            obj.findMin();
            break;
        case 4:
            obj.findSum();
            break;
        case 5:
            obj.findAverage();
            break;
        case 6:
            obj.searchElement();
            break;
        case 7:
            System.out.println("Exiting Program...");
            break;
        default:
            System.out.println("Invalid choice!");
    }

} while (choice != 7);
}
}

```

Assignment No. 6 Program

Development of a Java program to implement a hotel room booking system using two-dimensional arrays.

```
import java.util.Scanner;

class HotelRoom {
    int rooms[][];

    HotelRoom(int floors, int
roomsPerFloor) {

        rooms = new int[floors][roomsPerFloor]; // 0 = Available, 1 = Booked
    }

    void displayRooms() {
        System.out.println("\nRoom Status (0 = Available, 1 = Booked)");
        for (int i = 0; i < rooms.length; i++) {
            System.out.print("Floor " + (i + 1) + ": ");
            for (int j = 0;
j < rooms[i].length; j++) {
                System.out.print(rooms[i][j] + " ");
            }
            System.out.println();
        }
    }

    void bookRoom() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Floor Number: ");
        int
floor = sc.nextInt() - 1;
        System.out.print("Enter Room Number: ");
        int room = sc.nextInt() - 1;
        if (floor >= 0 && floor < rooms.length && room >= 0 && room < rooms[floor].length) {
            if (rooms[floor][room] == 0) {
                rooms[floor][room] = 1;
                System.out.println("Room successfully booked!");
            } else {
                System.out.println("Room already booked!");
            }
        } else {
            System.out.println("Invalid floor or room number!");
        }
    }
}
```

```

    }
}

public class HotelRoomSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of floors: ");
        int floors = sc.nextInt();

        System.out.print("Enter number of rooms per floor: ");
        int roomsPerFloor = sc.nextInt();

        HotelRoom hotel = new HotelRoom(floors, roomsPerFloor); int
choice;

        do {
            System.out.println("\n--- Hotel Room Booking Menu ---");
            System.out.println("1. View Rooms");
            System.out.println("2. Book a Room");
            System.out.println("3. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
case 1:
            hotel.displayRooms();
            break;
            case 2:
            hotel.bookRoom();
            break;
            case 3:
            System.out.println("Thank you! Exiting system.");
            break;
            default:
            System.out.println("Invalid choice!");
            }

        } while (choice != 3);
    }
}

```

Group B Assignments

Assignment No. 1

Implementation of a Java program to demonstrate Single Inheritance.

```
class Animal {
void eat() {
    System.out.println("Animal is eating.");
}
void sleep() {
    System.out.println("Animal is sleeping.");
}
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking.");
    }
    // Subclass calling superclass methods
    void showActions() {
        System.out.println("--- Dog Actions ---");
        eat();           // calling superclass method
        sleep();          // calling superclass method
        bark();           // subclass method
    }
}
public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.showActions();
    }
}
```

Assignment No. 2

Implementation of a Java program to demonstrate Polymorphism.

```
interface Shape {
    void draw(); // abstract method
    double area(); // abstract method
}

class Circle implements Shape
{
    double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    public void
draw() {
        System.out.println("Drawing a Circle");
    }
    public double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements
Shape {
    double length, width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width =
width;
    }
    public void
draw() {
        System.out.println("Drawing a Rectangle");
    }
    public double area()
{
        return length *
width;
    }
}

public class InterfacePolymorphismDemo {
    public static void main(String[] args) {
        Shape s;
        // interface reference
        s = new Circle(4.5); //
pointing to Circle object
        s.draw();
        System.out.println("Circle Area: " + s.area());
        System.out.println();
        s = new Rectangle(5, 8); //
pointing to Rectangle object
```

```
s.draw();  
System.out.println("Rectangle Area: " + s.area());  
}  
}
```

Assignment No. 3

Implementation of a Java program
to create an Abstract class.

```
//Abstract superclass abstract
class Vehicle {    // Abstract
method (no body)    abstract
void start();    // Normal
method    void stop() {
    System.out.println("Vehicle has stopped.");
}
}

//Subclass implementing abstract method class
Car extends Vehicle {
    // Providing implementation of abstract method
void start() {
    System.out.println("Car is starting with a key ignition.");
}
}

//Main class public class AbstractClassDemo
{    public static void main(String[] args)
{
    // Using parent reference and child object
    Vehicle v = new Car();
    v.start(); // Calls implemented method in Car
    v.stop(); // Calls normal method from Vehicle
}
}
```

Assignment No. 4

Development of a Java application to simulate an ATM machine using exception handling mechanism.

```
import
java.util.Scanner;
class ATM {
private double balance;
ATM(double initialBalance)
{
    balance = initialBalance;
}
void checkBalance() {
    System.out.println("Current Balance: ₹" + balance);
}
void deposit(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Deposit amount must be positive.");
    }
    balance
    += amount;
    System.out.println("₹" + amount + " deposited successfully.");
}
void withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Withdrawal amount must be positive.");
    }
    if (amount > balance) {
        throw new
        ArithmeticException("Insufficient funds.");
    }
    balance
    -= amount;
    System.out.println("₹" + amount + " withdrawn successfully.");
} }
public class ATMSimulator {
    public
    static void main(String[] args) {
        Scanner
        sc = new Scanner(System.in);
        ATM atm = new
        ATM(5000); // initial balance
        int choice;
        do {
            System.out.println("\n----- ATM MENU -----");
            System.out.println("1. Check Balance");
```

```

        System.out.println("2. Deposit Money");
        System.out.println("3. Withdraw Money");
System.out.println("4. Exit");
        System.out.print("Enter your choice: ");
try {
            choice = sc.nextInt();
switch (choice) {
            case 1:
atm.checkBalance();
                break;
case 2:
                System.out.print("Enter deposit amount: ");
double depositAmount = sc.nextDouble();
atm.deposit(depositAmount);
                break;
case 3:
                System.out.print("Enter withdrawal amount: ");
double withdrawAmount = sc.nextDouble();
atm.withdraw(withdrawAmount);
                break;
case 4:
                System.out.println("Thank you for using ATM.");
break;
                default:
                    throw new IllegalArgumentException("Invalid menu choice.");
            }

        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Error: Invalid input. Please enter numbers only.");
        } finally {
            System.out.println("Transaction completed.\n");
        }
    } while (true);
}
}

```

Assignment No. 5

Development of a Java application to simulate an online shopping system using exception handling.

```
import java.util.Scanner;

class ShoppingCart {    private double totalPrice = 0;
void addItem(String itemName, double price, int quantity) {
if (price <= 0 || quantity <= 0) {
        throw new ArithmeticException("Price and quantity must be greater than
zero.");    }    double itemTotal = price * quantity;
totalPrice += itemTotal;

        System.out.println(itemName + " added to cart. Item total: ₹" +
itemTotal);
    }    void
showTotal() {
        System.out.println("Total cart value: ₹" + totalPrice);
    }    void processPayment(double payment) {        if (payment <
totalPrice) {            throw new ArithmeticException("Insufficient
payment amount.");
        }        double change = payment -
totalPrice;

        System.out.println("Payment successful.");
System.out.println("Change returned: ₹" + change);
totalPrice = 0; // cart cleared
    } } public class OnlineShoppingSystem
{    public static void main(String[]
args) {        Scanner sc = new
Scanner(System.in);

        ShoppingCart cart = new ShoppingCart();
        int choice;

do {

        System.out.println("\n--- ONLINE SHOPPING MENU ---");
        System.out.println("1. Add item to cart");
        System.out.println("2. Show total price");
```

```

        System.out.println("3. Make payment");
        System.out.println("4. Exit");
        System.out.print("Enter your choice: ");
try {
            choice =
Integer.parseInt(sc.nextLine());
            switch
(choice) {
                case 1:
                    System.out.print("Enter item name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter item price: ");
double price = Double.parseDouble(sc.nextLine());
                    System.out.print("Enter quantity: ");
int qty =
Integer.parseInt(sc.nextLine());
                    cart.addItem(name, price, qty);
                    break;
                case 2:
                    cart.showTotal();
break;
                case 3:
                    System.out.print("Enter payment amount: ");
double payment = Double.parseDouble(sc.nextLine());
                    cart.processPayment(payment);
                    break;
                case 4:
                    System.out.println("Thank you for shopping!");
return;
                default:
                    System.out.println("Invalid choice.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid numeric input.");
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Transaction completed.");
        }
    } while (true);
}
}

```

Assignment No. 6

Development of a Java
application to monitor
stock prices in real-time
using
multithreading and
synchronization.

```
import java.util.Random;

class Stock {
    private double price;

    // synchronized method to update price
    synchronized void setPrice(double price) {
        this.price = price;
    }

    // synchronized method to read price
    synchronized double getPrice() {
        return price;
    }
}

// Thread 1: Fetch stock price (simulated API)
class PriceFetcher extends Thread {
    private Stock stock;

    PriceFetcher(Stock stock) {
        this.stock = stock;
    }

    public void run() {
        Random random = new Random();

        for (int i = 1; i <= 5; i++) {
            try {
                double newPrice = 100 + random.nextInt(50);
                stock.setPrice(newPrice);
                System.out.println("Fetched stock price: ₹" + newPrice);

                Thread.sleep(1000); // simulate API delay
            } catch (InterruptedException e) {
                System.out.println("Fetcher thread interrupted");
            }
        }
    }
}
```

```

    }
}

// Thread 2: Display stock price class
PriceDisplay extends Thread {
private Stock stock;
PriceDisplay(Stock stock) {
this.stock = stock;
    }    public void run() {
for (int i = 1; i <= 5; i++) {
try {
        System.out.println("Displayed stock price: ₹" +
stock.getPrice());
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        System.out.println("Display thread interrupted");
    }
}
} } public class StockMonitor {
public static void main(String[] args) {
    Stock stock = new Stock();

    PriceFetcher fetcher = new PriceFetcher(stock);
    PriceDisplay display = new PriceDisplay(stock);

    fetcher.start();
    display.start();
    try {
        fetcher.join();
        display.join();
    } catch (InterruptedException e) {
        System.out.println("Main thread interrupted");
    }
    System.out.println("Stock monitoring completed.");
}
}

```

Assignment No. 7

Development of a multi-threaded Java application to simulate a basic chat system using thread control methods.

```
class ChatUser extends Thread {
    private String userName;
    private volatile boolean paused = false;
    private volatile boolean stopped = false;
    ChatUser(String name) {
        this.userName = name;
    }
    // Suspend simulation
    synchronized void pauseChat() {
        paused = true;
    }
    // Resume simulation
    synchronized void resumeChat() {
        paused = false;        notify();
    }
    // Stop simulation
    void stopChat() {
        stopped = true;
    }
    @Override    public void run() {
        int msgCount = 1;        try {
            while (!stopped && msgCount <= 5) {
                synchronized (this) {
                    while (paused) {
                        wait(); // thread suspension
                    }
                }
                System.out.println(userName + " sending message " + msgCount);
                msgCount++;
                Thread.sleep(1000); // simulate delay
            }
        }
    }
}
```

```

        } catch (InterruptedException e) {
            System.out.println(userName + " interrupted.");
        }
        System.out.println(userName + " chat ended.");
    } } public class ChatSystem {
public static void main(String[] args) {
    ChatUser user1 = new ChatUser("Alice");
    ChatUser user2 = new ChatUser("Bob");
    // Setting priorities
user1.setPriority(Thread.MAX_PRIORITY); // high-priority user
user2.setPriority(Thread.MIN_PRIORITY);      user1.start();
user2.start();          // Check thread status
    System.out.println("Is Alice alive? " + user1.isAlive());
System.out.println("Is Bob alive? " + user2.isAlive());
    try {
        Thread.sleep(2000);
        // Suspend Bob
        System.out.println("Pausing Bob...");
user2.pauseChat();
        Thread.sleep(2000);
        // Resume Bob
        System.out.println("Resuming Bob...");
user2.resumeChat();      Thread.sleep(2000);
        // Stop Alice
        System.out.println("Stopping Alice...");
user1.stopChat();        // Ensure threads
        finish          user1.join();
user2.join();
    } catch (InterruptedException e) {
        System.out.println("Main thread interrupted.");
    }
    System.out.println("Chat system terminated.");
}
}

```

PART C

Assignment 1

Development of a Java application to manage employee information using JDBC and database connectivity

```
Main.java import java.util.Scanner;

    public class Main {        public static void
main(String[] args) {            Scanner sc =
new Scanner(System.in);        EmployeeDAO
dao = new EmployeeDAO();        while
(true) {

        System.out.println("\n1. Add Employee");
        System.out.println("2. View Employees");
        System.out.println("3. Update Employee");
        System.out.println("4. Delete Employee");
        System.out.println("5. Exit");

System.out.print("Enter choice: ");

int choice = sc.nextInt();        switch
(choice) {            case 1:

                sc.nextLine();
                System.out.print("Name: ");
                String name = sc.nextLine();

                System.out.print("Department: ");
                String dept = sc.nextLine();

                System.out.print("Salary: ");
                double salary = sc.nextDouble();

                dao.addEmployee(new Employee(name, dept, salary));

break;

                case 2:

dao.viewEmployees();

break;

                case 3:
```

```

        System.out.print("Enter ID: ");
        int id = sc.nextInt();

        System.out.print("New Salary: ");
        double newSalary = sc.nextDouble();
        dao.updateEmployee(id,
newSalary);
        break;
        case
4:
        System.out.print("Enter ID: ");
        int deleteId = sc.nextInt();
        dao.deleteEmployee(deleteId);
        break;
        case
5:
        System.out.println("Exiting...");
        sc.close();
        return;

default:
        System.out.println("Invalid choice!");
    }
}
}
}
}
}

```

```

Employee.java public class
Employee {    private int id;
private String name;
private String department;
private double salary;

    public Employee(String name, String department, double salary) {
this.name = name;
        this.department = department;
        this.salary = salary;
    }
}

```

```

    public Employee(int id, String name, String department, double salary) {
this.id = id;          this.name = name;          this.department =
department;          this.salary = salary;
    }    public int getId() { return id; }    public
String getName() { return name; }    public String
getDepartment() { return department; }    public double
getSalary() { return salary; }
}

```

EmployeeDAO.java import

```

java.sql.*;

public class EmployeeDAO {    // Add
Employee    public void
addEmployee(Employee emp) {
    String query = "INSERT INTO employees (name, department, salary) VALUES (?, ?,
?);";    try (Connection conn = DBConnection.getConnection();
PreparedStatement pst = conn.prepareStatement(query)) {
pst.setString(1, emp.getName());    pst.setString(2,
emp.getDepartment());    pst.setDouble(3,
emp.getSalary());    pst.executeUpdate();
        System.out.println("Employee added successfully!");
    } catch (SQLException e) {
        System.out.println("Error adding employee");
        e.printStackTrace();
    }
}

// View Employees
public void viewEmployees() {
    String query = "SELECT * FROM employees";    try
(Connection conn = DBConnection.getConnection());
        PreparedStatement pst = conn.prepareStatement(query);
ResultSet rs = pst.executeQuery() {    while (rs.next())
{
        System.out.println(
rs.getInt("id") + " | " +    rs.getString("name")
+ " | " +    rs.getString("department") + " | " +
rs.getDouble("salary")

```

```

        );
    }

    } catch (SQLException e) {
        System.out.println("Error fetching employees");
        e.printStackTrace();
    }
}

// Update Employee    public void
updateEmployee(int id, double salary) {
    String query = "UPDATE employees SET salary = ? WHERE id = ?";
    try (Connection conn =
DBConnection.getConnection();
        PreparedStatement pst = conn.prepareStatement(query)) {
        pst.setDouble(1,
salary);          pst.setInt(2,
id);

        int rows =
pst.executeUpdate();
        if (rows > 0)
{
            System.out.println("Employee updated successfully!");
        } else {
            System.out.println("Employee not found.");
        }

    } catch (SQLException e) {
        System.out.println("Error updating employee");
        e.printStackTrace();
    }
}

// Delete Employee    public void
deleteEmployee(int id) {
    String query = "DELETE FROM employees WHERE id = ?";
    try (Connection conn =
DBConnection.getConnection());

```

```

        PreparedStatement pst = conn.prepareStatement(query)) {
            pst.setInt(1,
id);

            int rows =
pst.executeUpdate();
            if (rows > 0)
{
                System.out.println("Employee deleted successfully!");
            } else {
                System.out.println("Employee not found.");
            }
        } catch (SQLException e) {
            System.out.println("Error deleting employee");
            e.printStackTrace();
        }
    }
}

```

DBConnection.java import

java.sql.Connection; import

java.sql.DriverManager; import

java.sql.SQLException;

```

public class DBConnection {    private static final String URL =
"jdbc:mysql://localhost:3306/employee_db"; private static final String USER =
"root";    private static final String PASSWORD = "root";

```

```

        public static Connection getConnection() {            try {
return DriverManager.getConnection(URL, USER, PASSWORD);
            } catch (SQLException e) {
                System.out.println("Database connection failed!");
                e.printStackTrace();
return null;
            }
        }
    }
}

```

Sample Output:

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 1

Name: Nikhil

Department: Manager

Salary: 56000.50

Employee added successfully!

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee 5. Exit

Enter choice: 1

Name: Pooja

Department: Developer

Salary: 45000.80

Employee added successfully!

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 1

Name: Saurabh

Department: Tester

Salary: 35000.90

Employee added successfully!

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 2

```
1 | nikita | comp | 12000.0
2 | Nikhil | Manager | 56000.5
3 | Pooja | Developer | 45000.8
4 | Saurabh | Tester | 35000.9
```

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 3

Enter ID: 2

New Salary: 58000.90

Employee updated successfully!

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter choice: 2

```
1 | nikita | comp | 12000.0
2 | Nikhil | Manager | 58000.9
3 | Pooja | Developer | 45000.8
4 | Saurabh | Tester | 35000.9
```

1. Add Employee
2. View Employees

3. Update Employee

4. Delete Employee

5. Exit

Enter choice: 4

Enter ID: 2

Employee deleted successfully!

1. Add Employee

2. View Employees

3. Update Employee

4. Delete Employee

5. Exit

Enter choice: 2

1 | nikita | comp | 12000.0

3 | Pooja | Developer | 45000.8

4 | Saurabh | Tester | 35000.9

1. Add Employee

2. View Employees

3. Update Employee

4. Delete Employee 5. Exit

Enter choice: 5

Exiting...

Assignment 2

Development of a Java application to manage student information using JDBC with transaction management.

```
Main.java import java.util.Scanner;

public class Main {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
    StudentDAO dao = new StudentDAO();
    while (true) {
        System.out.println("\n--- Student Management ---");
        System.out.println("1. Add Student");
        System.out.println("2. View Students");
        System.out.println("3. Update Student");
        System.out.println("4. Delete Student");
        System.out.println("5. Exit");

        System.out.print("Enter choice: ");
int choice = sc.nextInt();
        switch (choice) {
case 1:
            sc.nextLine();
            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Email: ");
            String email = sc.nextLine();

            System.out.print("Course: ");
            String course = sc.nextLine();
            dao.addStudent(new Student(name, email, course));
break;

            case 2:
                dao.viewStudents();
break;
```

```

        case 3:
            System.out.print("Enter ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("New Course: ");
            String newCourse = sc.nextLine();
            dao.updateStudent(id, newCourse);

            break;

        case 4:
            System.out.print("Enter ID: ");
            int deleteId = sc.nextInt();
            dao.deleteStudent(deleteId);
            break;

        case 5:
            System.out.println("Exiting...");
            sc.close();
            return;

        default:
            System.out.println("Invalid choice!");
    }
}
}
}
}
}

```

```

Student.java public
class Student {
    private int id;
    private String name;
    private String email;
    private String course;

    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Student(int id, String name, String email, String
course) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }
}

```

```

    }    public int getId() { return id; }
public String getName() { return name; }
public String getEmail() { return email; }
public String getCourse() { return course; }
}

```

StudentDAO.java import

```

java.sql.*;

public class StudentDAO {

    // CREATE    public void
addStudent(Student student) {
    String query = "INSERT INTO students (name, email, course) VALUES (?, ?, ?)";
    try (Connection conn = DBConnection.getConnection()) {
conn.setAutoCommit(false); // Start transaction

        try (PreparedStatement pst = conn.prepareStatement(query)) {

            pst.setString(1, student.getName());
            pst.setString(2, student.getEmail());
pst.setString(3, student.getCourse());

            pst.executeUpdate();

            conn.commit(); // Commit transaction
            System.out.println("Student added successfully!");

        } catch (SQLException e) {
conn.rollback(); // Rollback on error
System.out.println("Transaction rolled back!");
            e.printStackTrace();
        }

    } catch (SQLException e) {
        System.out.println("Database error!");
        e.printStackTrace();
    }
}

```

```

    }
}

// READ    public void
viewStudents() {
    String query = "SELECT * FROM students";
    try (Connection conn = DBConnection.getConnection());
        PreparedStatement pst = conn.prepareStatement(query);
        ResultSet rs = pst.executeQuery()) {
        while (rs.next()) {
System.out.println(
rs.getInt("id") + " | " +
rs.getString("name") + " | " +
rs.getString("email") + " | " +
rs.getString("course")
                );
        }
    } catch (SQLException e) {
        System.out.println("Error fetching students");
        e.printStackTrace();
    }
}

// UPDATE    public void updateStudent(int id,
String course) {
    String query = "UPDATE students SET course = ? WHERE id = ?";
    try (Connection conn = DBConnection.getConnection()) {
conn.setAutoCommit(false);
        try (PreparedStatement pst = conn.prepareStatement(query)) {
            pst.setString(1, course);
pst.setInt(2, id);
            int rows = pst.executeUpdate();
            if (rows > 0) {
conn.commit();

                System.out.println("Student updated!");
            }
        }
    }
}

```

```

        } else {
conn.rollback();

        System.out.println("Student not found!");
        }

    } catch (SQLException e) {
conn.rollback();

        System.out.println("Update failed. Rolled back.");
        e.printStackTrace();
    }

} catch (SQLException e) {
    System.out.println("Database error!");
    e.printStackTrace();
}
}

// DELETE    public void
deleteStudent(int id) {
    String query = "DELETE FROM students WHERE id = ?";
    try (Connection conn = DBConnection.getConnection()) {
conn.setAutoCommit(false);

        try (PreparedStatement pst = conn.prepareStatement(query)) {
            pst.setInt(1, id);
            int rows = pst.executeUpdate();
            if (rows > 0) {
conn.commit();

                System.out.println("Student deleted!");
            } else {
conn.rollback();

                System.out.println("Student not found!");
            }

        } catch (SQLException e) {
conn.rollback();

            System.out.println("Delete failed. Rolled back.");

```

```

        e.printStackTrace();
    }

    } catch (SQLException e) {
        System.out.println("Database error!");
        e.printStackTrace();
    }
}

}

DBConnection.java import
java.sql.Connection; import
java.sql.DriverManager; import
java.sql.SQLException;

public class DBConnection {    private static final String URL =
"jdbc:mysql://localhost:3306/student_db";    private static final String USER =
"root";    private static final String PASSWORD = "root";

    public static Connection getConnection() throws SQLException {
return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

Sample Output:

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 1

Name: Neha

Email: neha@gmail.com

Course: BTech

Student added successfully!

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 1

Name: Amit

Email: amit@gmail.com

Course: Bcom

Student added successfully!

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 1

Name: Akshay

Email: akshay@gmail.com

Course: BBA

Student added successfully!

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 2

- 2 | sarthak | sarthak@gmail.com | pps
- 3 | Neha | neha@gmail.com | BTech
- 4 | Amit | amit@gmail.com | Bcom
- 5 | Akshay | akshay@gmail.com | BBA

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student 5. Exit

Enter choice: 3

Enter ID: 2

New Course: MBA

Student updated!

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 2

- 2 | sarthak | sarthak@gmail.com | MBA
- 3 | Neha | neha@gmail.com | BTech
- 4 | Amit | amit@gmail.com | Bcom

5 | Akshay | akshay@gmail.com | BBA

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 4

Enter ID: 2

Student deleted!

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 2

- 3 | Neha | neha@gmail.com | BTech
- 4 | Amit | amit@gmail.com | Bcom
- 5 | Akshay | akshay@gmail.com | BBA

--- Student Management ---

1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Exit

Enter choice: 5 Exiting...